Lecture 5

More Power and Early Stopping in A/B Testing

(Please, sign in on iClicker)

Today's Learning Objectives

1. Comment on nuances of randomized experiments that arise specifically for website optimization problems.
2. Determine an adequate sample size (i.e., experiment duration) for such problems.
3. Apply clever plotting arrangements to communicate power analyses.
4. Relate the danger of "early stopping" to experiments in A/B testing.
5. Analyze the advantages and disadvantages of aggressive and principled peekings in A/B testing.

Outline

1. Sample Size Computations in Plots
2. Sample Size Computations with Click-through Rates
3. Early Stopping in A/B Testing

1. Sample Size Computations in Plots

We already introduced the R function `pwr.t.test()` from library `pwr` to compute sample sizes in an A/B testing framework where the response is continuous.
Moreover, we generated a table with different scenarios of power $1 - \beta$, significance level α, and effect difference δ.
We also introduced a handy Power Analysis plotting arrangement in `lecture4`.
Let us explore other plotting alternatives!

1.1. Revisiting the Example

We want to do an A/B testing to see if a tweak (version B) of our existing site (version A) promotes site visitors to stay longer.

> Heads-up: The boss will not sign off on this without some certainty around how many visitors will be randomized to either version A or B.

Additionally

- We know that CURRENTLY, around 95% of the visitors stay between 30 seconds and 2 minutes.
- We would permanently switch to the tweaked site if we were convinced it improved visit length by 3% (or more!).

Main Statistical Inquiry

Given the problem's setup, how large should we set up our sample size so we can detect this difference (if there is any)?

1.2. Using Function `pwr.t.test()`

We will not go into the specific details on the use of the function `pwr.t.test()` and the computations of $\mu_A = 1.25$ and $\sigma = 0.383$ min.
Nonetheless, we need to set up the following values for computing different sample sizes:

- A single value for $\alpha = 0.05$ (i.e., the default significance level).
- $1 - \beta$ (power): $0.70, 0.80$, and 0.90. The larger the power, the less prone we will be to commit type II error.
- Increases in the response of interest with $\delta = 0.0375, 0.0625, 0.0875, 0.1125, 0.1375, 0.1625$ in treatment B with respect to treatment A (i.e., increases of 3, 5, 7, 9, 11, and 13%).

As previously done in `lecture4`

We need to prepare a grid with all the possible combinations for `alpha`, `power`, and `delta`.

R Code Start Over Run Code

```r
# Let us obtain all possible setting combinations
alpha <- 0.05
power <- c(0.7, 0.8, 0.9)
delta <- c(0.0375, 0.0625, 0.0875, 0.1125, 0.1375, 0.1625)
experimental_grid <- expand.grid(alpha, power, delta)
colnames(experimental_grid) <- c("alpha", "power", "delta")
# Adding a column for the overall sample size n
experimental_grid$overall_n <- NA
```

Experimental grid

R Code Start Over Run Code

```r
head(experimental_grid, n=15)
```

```
   alpha power  delta overall_n
1   0.05   0.7 0.0375        NA
2   0.05   0.8 0.0375        NA
3   0.05   0.9 0.0375        NA
4   0.05   0.7 0.0625        NA
5   0.05   0.8 0.0625        NA
6   0.05   0.9 0.0625        NA
7   0.05   0.7 0.0875        NA
8   0.05   0.8 0.0875        NA
9   0.05   0.9 0.0875        NA
10  0.05   0.7 0.1125        NA
11  0.05   0.8 0.1125        NA
12  0.05   0.9 0.1125        NA
13  0.05   0.7 0.1375        NA
14  0.05   0.8 0.1375        NA
15  0.05   0.9 0.1375        NA
```

Then

We use the function `pwr.t.test()` with all the possible combinations from `experimental_grid`. We also need to set up the value for `sigma`. Recall the elements of `pwr.t.test()`:

- `d`: Desired effect size (δ divided by σ).
- `sig.level`: Chosen α.
- `power`: Chosen power $1 - \beta$.
- `type`: Type of test (in the A/B testing context we have a `two.sample` test).
- `alternative`: Our hypothesis test is one-sided, thus `greater`.

Compute sample size

R Code Start Over Run Code

```r
sigma <- 0.383
for (i in 1:nrow(experimental_grid)) {
  # Using pwr.t.test() function
  sample_size <- tidy(pwr.t.test(
    d = experimental_grid[i, "delta"] / sigma,
    sig.level = experimental_grid[i, "alpha"],
    power = experimental_grid[i, "power"],
    type = "two.sample",
    alternative = "greater"
  ))

  # Recording overall sample size
  experimental_grid[i, "overall_n"] <- 2 * ceiling(sample_size$n)
}
```

Overall sample size

R Code Start Over Run Code

```
experimental_grid <- experimental_grid %>% mutate(power = as.factor(power))
head(experimental_grid,n=15)
```

```
   alpha power  delta overall_n
1   0.05   0.7 0.0375      1966
2   0.05   0.8 0.0375      2582
3   0.05   0.9 0.0375      3576
4   0.05   0.7 0.0625       710
5   0.05   0.8 0.0625       932
6   0.05   0.9 0.0625      1288
7   0.05   0.7 0.0875       362
8   0.05   0.8 0.0875       476
9   0.05   0.9 0.0875       658
10  0.05   0.7 0.1125       220
11  0.05   0.8 0.1125       288
12  0.05   0.9 0.1125       400
13  0.05   0.7 0.1375       148
14  0.05   0.8 0.1375       194
15  0.05   0.9 0.1375       268
```

1.3. Plotting Results

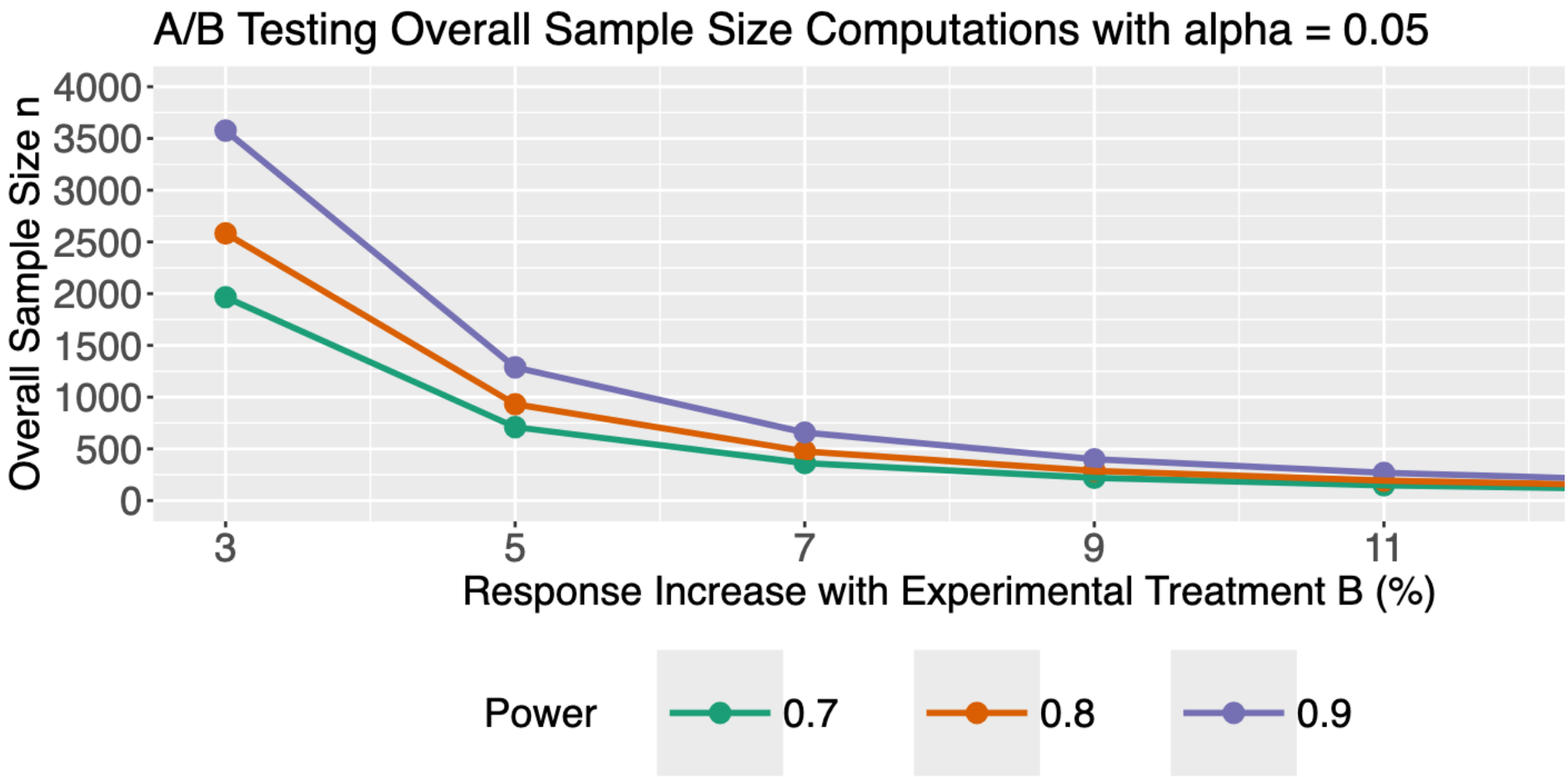
R Code Start Over Run Code

```r
t_sample_size_plot <- ggplot(experimental_grid, aes(x = delta, y = overall_n,
  geom_point(size = 4) +
  geom_line(linewidth = 1) +
  theme(
    plot.title = element_text(size = 20),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    strip.text = element_text(size = 18),
    legend.position = "bottom",
    legend.title = element_text(size = 18, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 18, margin = margin(r = 1, unit = "cm")
    legend.key.size = unit(2, "cm")
  ) +
  labs(x = "Response Increase with Experimental Treatment B (%)", y = "Overal
  coord_cartesian(xlim = c(0.0375, 0.1625), ylim = c(0, 4000)) +
  scale_y_continuous(breaks = seq(0, 4000, 500)) +
  scale_x_continuous(breaks = delta, labels = c(3, 5, 7, 9, 11, 13)) +
  labs(colour = "Power") +
  ggtitle("A/B Testing Overall Sample Size Computations with alpha = 0.05") +
  scale_color_brewer(palette = "Dark2")
```

Sample size plot

R Code Start Over Run Code

```
t_sample_size_plot
```


A/B Testing Overall Sample Size Computations with alpha = 0.05
Overall Sample Size n
4000
3500
3000
2500
2000
1500
1000
500
0
3
5
7
9
11
Response Increase with Experimental Treatment B (%)
Power
0.7
0.8
0.9

iClicker Question

If we cannot anticipate a large percentage response increase with our experimental version B, compared to control treatment A, what can we state about the overall sample size?

A. If we cannot ensure a large percentage response increase with our experimental version B in mean duration visits, compared to control treatment A, our A/B testing will require more experimental units.

B. If we cannot ensure a large percentage response increase with our experimental version B in mean duration visits, compared to control treatment A, our A/B testing will require less experimental units.

Answer

Using this plot, it is easier to say something about our existing options to set up the overall sample size n for the A/B testing.
Note that we use a standard value for controlling false positives, which is $\alpha = 0.05$.
if we CANNOT ensure a large percentage increase with our experimental version B in mean duration visits, (compared to control treatment A) our A/B testing will require more experimental units.
This matter gets even more critical if we need a larger power!

1.4. Alternative Plotting

Suppose your boss tells you there is available budget for an A/B testing for an overall simple size up to $n = 1000$.
You can use `pwr.t.test()` to compute the corresponding levels of power with the following settings:
Overall sample sizes from 100 to 1000 increasing by 100.
A single value for $\alpha = 0.05$ (i.e., the default significance level).
Increases in the response of interest with
$\delta = 0.0375, 0.0625, 0.0875, 0.1125, 0.1375, 0.1625$ in treatment B with respect to treatment A (i.e., increases of 3, 5, 7, 9, 11, and 13%).

Let us obtain all possible setting combinations

R Code Start Over Run Code

```r
sample_size_seq <- seq(100, 1000, by = 100)
experimental_grid_alt <- expand.grid(alpha, sample_size_seq, delta)
colnames(experimental_grid_alt) <- c("alpha", "sample_size", "delta")
# Adding a column for power
experimental_grid_alt$power <- NA
```

Code in R!

R Code Start Over Run Code

```r
head(experimental_grid_alt, n=15)
```

```
   alpha sample_size  delta power
1   0.05         100 0.0375    NA
2   0.05         200 0.0375    NA
3   0.05         300 0.0375    NA
4   0.05         400 0.0375    NA
5   0.05         500 0.0375    NA
6   0.05         600 0.0375    NA
7   0.05         700 0.0375    NA
8   0.05         800 0.0375    NA
9   0.05         900 0.0375    NA
10  0.05        1000 0.0375    NA
11  0.05         100 0.0625    NA
12  0.05         200 0.0625    NA
13  0.05         300 0.0625    NA
14  0.05         400 0.0625    NA
15  0.05         500 0.0625    NA
```

`pwr.t.test` function in R!

We use the function `pwr.t.test()` with all the possible combinations from `experimental_grid_alt`. We use the same value for `sigma`. The elements of `pwr.t.test()` will be:

- `d`: Desired effect size (δ divided by $\sigma = 0.383$).
- `sig.level`: Chosen α.
- `n`: Number of experimental units PER TREATMENT.
- `type`: Type of test (in the A/B testing context we have a `two.sample` test).
- `alternative`: Our hypothesis test is one-sided, thus `greater`.

Code in R to compute power!

R Code Start Over Run Code

```
for (i in 1:nrow(experimental_grid_alt)) {
  # Using pwr.t.test() function
  sample_size <- tidy(pwr.t.test(
    d = experimental_grid_alt[i, "delta"] / sigma,
    sig.level = experimental_grid_alt[i, "alpha"],
    n = experimental_grid_alt[i, "sample_size"] / 2, # We divide over 2 given
    type = "two.sample",
    alternative = "greater"
  ))

  # Recording power
  experimental_grid_alt[i, "power"] <- sample_size$power
}
```

Power vs Sample size

R Code Start Over Run Code

```r
head(experimental_grid_alt, n = 15)
```

```
   alpha sample_size  delta     power
1   0.05         100 0.0375 0.1232954
2   0.05         200 0.0375 0.1698191
3   0.05         300 0.0375 0.2121911
4   0.05         400 0.0375 0.2522567
5   0.05         500 0.0375 0.2905908
6   0.05         600 0.0375 0.3274205
7   0.05         700 0.0375 0.3628448
8   0.05         800 0.0375 0.3969099
9   0.05         900 0.0375 0.4296397
10  0.05        1000 0.0375 0.4610493
11  0.05         100 0.0625 0.2019848
12  0.05         200 0.0625 0.3103352
13  0.05         300 0.0625 0.4071683
14  0.05         400 0.0625 0.4937073
15  0.05         500 0.0625 0.5702990
```

Code in R for plotting!

R Code Start Over Run Code

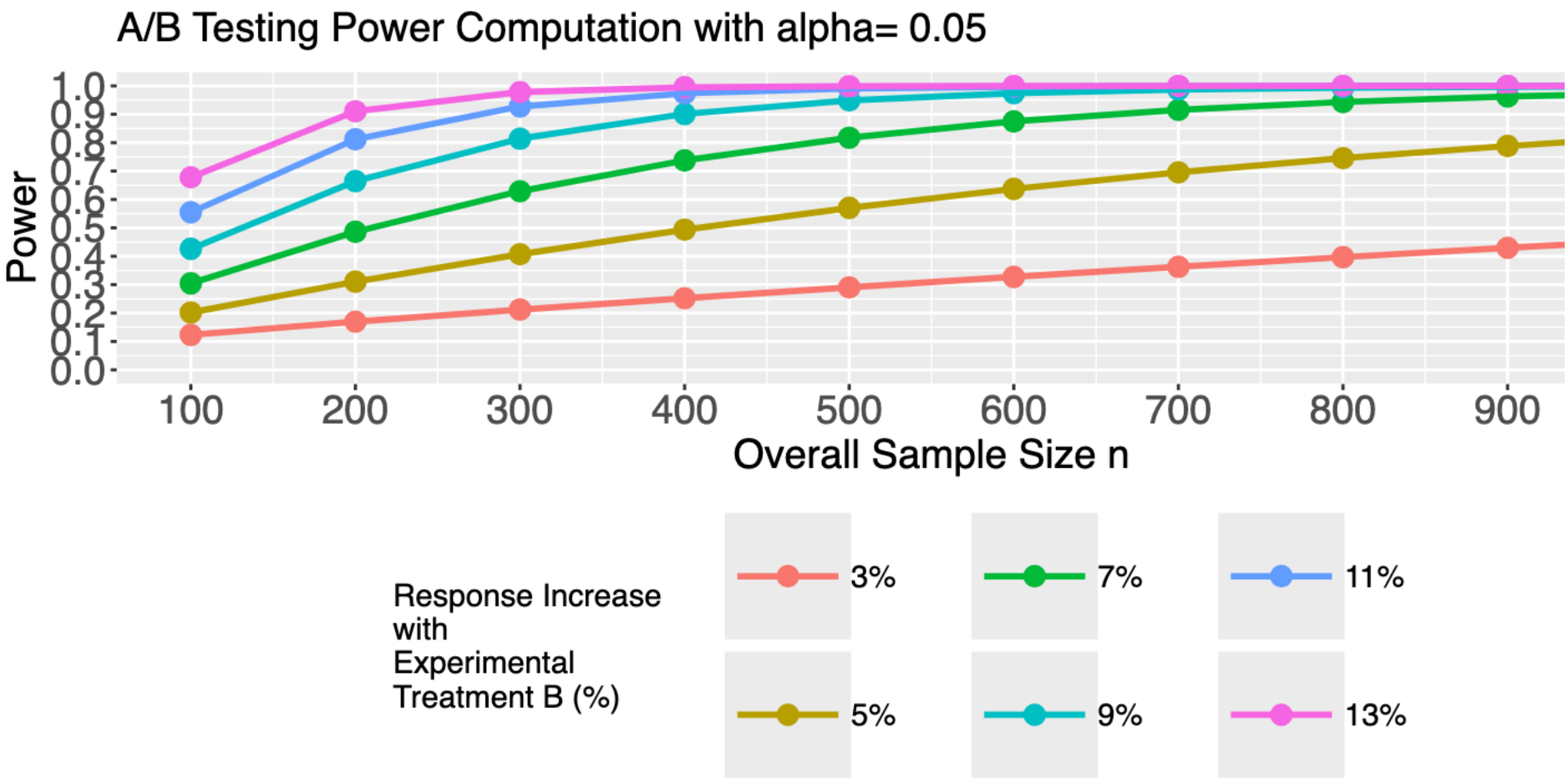
```r
t_power_plot <- ggplot(experimental_grid_alt, aes(x = sample_size, y = power,
  geom_point(size = 4) +
  geom_line(linewidth = 1) +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    strip.text = element_text(size = 18),
    legend.title = element_text(size = 14, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 14, margin = margin(r = 1, unit = "cm")
    legend.key.size = unit(2, "cm"),
    legend.position = "bottom"
  ) +
  labs(x = "Overall Sample Size n", y = "Power") +
  coord_cartesian(xlim = c(100, 1000), ylim = c(0, 1)) +
  scale_y_continuous(breaks = seq(0, 1, 0.1)) +
  scale_x_continuous(breaks = sample_size_seq) +
  labs(colour = "Response Increase\nwith\nExperimental\nTreatment B (%)") +
  ggtitle(expression("A/B Testing Power Computation with alpha= 0.05")) +
  scale_color_brewer(palette = "Dark2") +
  scale_colour_discrete(labels = c("3%", "5%", "7%", "9%", "11%", "13%"))
```

```
Scale for colour is already present.
Adding another scale for colour, which will replace the existing scale.
```

Power plot

R Code Start Over Run Code

```
t_power_plot
```


A/B Testing Power Computation with alpha= 0.05
Power
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
100
200
300
400
500
600
700
800
900
Overall Sample Size n
Response Increase with Experimental Treatment B (%)
3%
7%
11%
5%
9%
13%

iClicker Question

Use the previous Power Analysis plot to answer this question.

Answer TRUE or FALSE:

If we want to use a small overall sample size n, we will not struggle to find a decent power level with a response increase of 3% using our experimental treatment.

A. TRUE

B. FALSE

Answer

It is **FALSE**.

By checking the above Power Analysis plot, if we want to use a small sample size n, we will struggle to find a decent power level with a response increase of 3% using our experimental treatment.

iClicker Question

Use the previous Power Analysis plot to answer this question.

Answer TRUE or FALSE:

Beginning a response increase of 7% and beyond, our power levels get boosted as we increase our overall sample sizes. This is more evident starting $n = 400$.

A. TRUE

B. FALSE

Answer

It is TRUE.

2. Sample Size Computations with Click-through Rates

So far, our A/B testing sample size computations have been focused on continuous responses.
Nonetheless, recall we discussed that A/B testings could involve more than one outcome class.
Thus, let us explore sample size computations with a different outcome of interest: clickthrough rates (CTRs).

Response variable

Many A/B testing problems would involve a response which is binary. This response is measured as a CTR. Let us review this metric

$$\text{CTR}_j = \frac{\text{Total number of clicks on the button of interest in the } j\text{th treatment}}{\text{Total number of impressions on the } j\text{th treatment page}}$$

This response in the experiment is a proportion. Therefore:

- $\text{CTR}_A \in [0, 1]$ is the CTR of the control treatment A.
- $\text{CTR}_B \in [0, 1]$ is the CTR of the experimental treatment B.

Heads-up

In the context of a CTR, the sample size is equivalent to the OVERALL number of impressions in the A/B testing.
Since we are using a CTR as an outcome of interest, the function `pwr.t.test()` will not be helpful anymore.

> Hence, let us explore an alternative method.

A simple CTR example

Let us start with a simple CTR example for sample size computation.
We want to assess whether a tweak to our site would increase the CTR from $5/1000 = 0.005$ (i.e., 5 clicks per 1000 impressions in treatment A) to $7/1000 = 0.007$ (i.e., 7 clicks per 1000 impressions in treatment B).

R Code Start Over Run Code

```r
control_A_CTR <- 5/1000
tweak_B_CTR <- 7/1000
```

Effect size h for two proportions

Unlike a continuous response, we do not need to compute σ nor a mean for treatment A.

However, we need to compute the effect size h for two proportions (i.e., two CTRs in an A/B testing).

We will use the function `ES.h()` for this purpose. Its first argument will be the CTR for the experimental treatment B and and the second one for the control treatment A.

ES.h function in R!

R Code Start Over Run Code

```
CTR_effect_size <- ES.h(tweak_B_CTR, control_A_CTR)
round(CTR_effect_size, 3)
```

```
[1] 0.026
```

2.1. Arcsine Transformation for the Effect Size

We are computing the effect size h as an arcsine transformation:

$$\text{Effect Size } h = \left[2 \times \arcsin\left(\sqrt{\text{CTR}_B}\right)\right] - \left[2 \times \arcsin\left(\sqrt{\text{CTR}_A}\right)\right].$$

or:

$$\text{Effect Size } h = 2 \times \left[\arcsin\left(\sqrt{\text{CTR}_B}\right) - \arcsin\left(\sqrt{\text{CTR}_A}\right)\right].$$

What is this transformation graphically doing?

Code in R!

R Code Start Over Run Code

```r
effect_size_plot <- ggplot() +
  geom_function(colour = "blue", fun = ES.h, args = list(p2 = 0)) +
  xlab("CTR") +
  ylab("Transformation") +
  geom_segment(aes(x = 0.05, y = 0, xend = 0.05, yend = ES.h(0.05, 0)),
    colour = "red", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0.1, y = 0, xend = 0.1, yend = ES.h(0.1, 0)),
    colour = "red", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0, y = ES.h(0.05, 0), xend = 0.05, yend = ES.h(0.05, 0
    colour = "purple", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0, y = ES.h(0.1, 0), xend = 0.1, yend = ES.h(0.1, 0)),
    colour = "purple", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0.5, y = 0, xend = 0.5, yend = ES.h(0.5, 0)),
    colour = "red", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0.55, y = 0, xend = 0.55, yend = ES.h(0.55, 0)),
    colour = "red", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0, y = ES.h(0.5, 0), xend = 0.5, yend = ES.h(0.5, 0)),
    colour = "purple", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0, y = ES.h(0.55, 0), xend = 0.55, yend = ES.h(0.5
    colour = "purple", linetype = "dashed", linewidth = 1.5
```

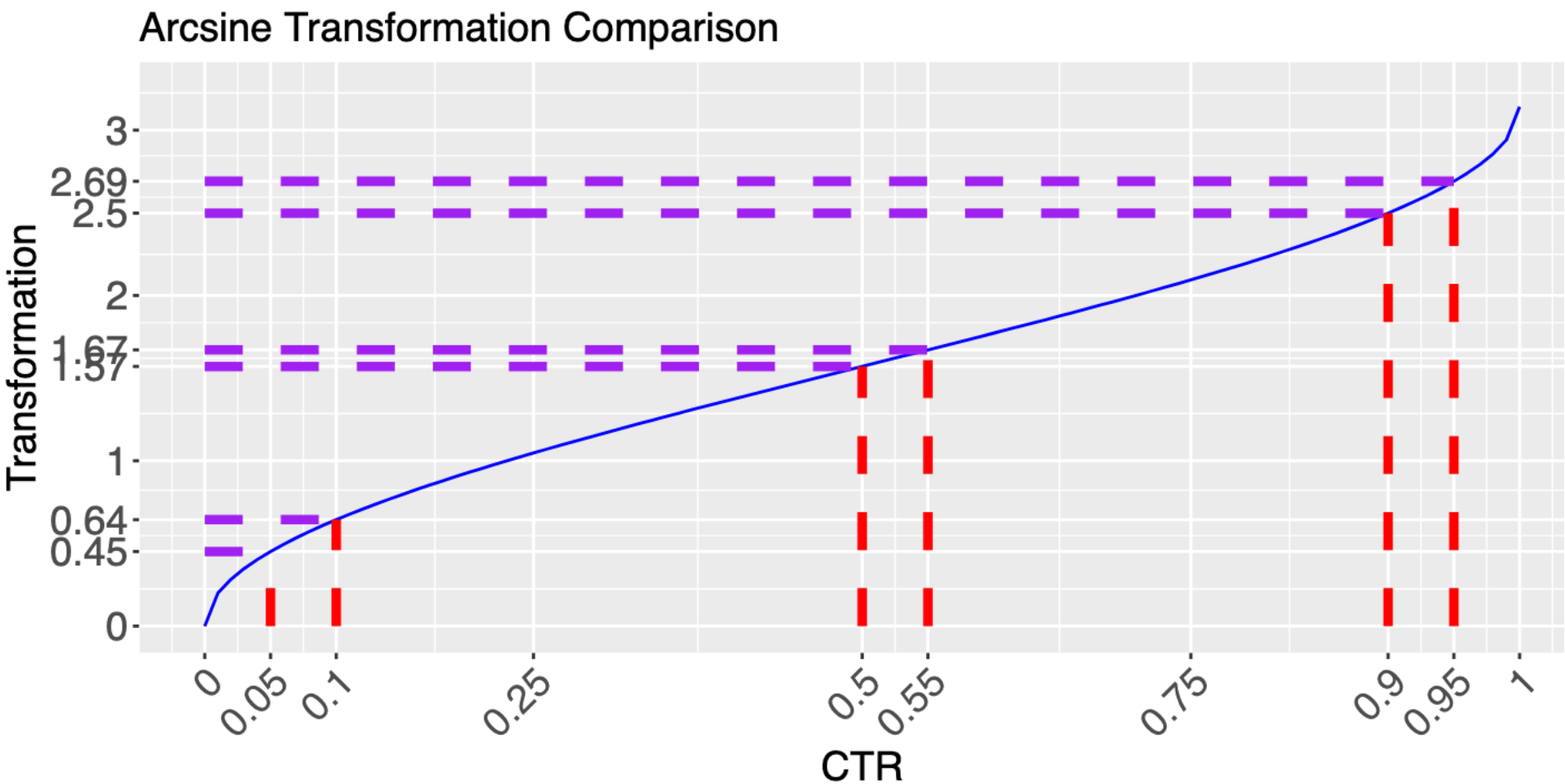
```
  ) +
  geom_segment(aes(x = 0.9, y = 0, xend = 0.9, yend = ES.h(0.9, 0)),
    colour = "red", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0.95, y = 0, xend = 0.95, yend = ES.h(0.95, 0)),
    colour = "red", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0, y = ES.h(0.9, 0), xend = 0.9, yend = ES.h(0.9, 0)),
    colour = "purple", linetype = "dashed", linewidth = 1.5
  ) +
  geom_segment(aes(x = 0, y = ES.h(0.95, 0), xend = 0.95, yend = ES.h(0.95, 0
    colour = "purple", linetype = "dashed", linewidth = 1.5
  ) +
  scale_x_continuous(
    limits = c(0, 1), breaks = c(0, 0.05, 0.1, 0.25, 0.5, 0.55, 0.75, 0.9, 0.
    labels = c(0, 0.05, 0.1, 0.25, 0.5, 0.55, 0.75, 0.9, 0.95, 1)
  ) +
  scale_y_continuous(
    limits = c(0, 3.25), breaks = c(0, ES.h(0.05, 0), ES.h(0.1, 0), 1, ES.h(0
    labels = c(0, round(ES.h(0.05, 0), 2), round(ES.h(0.1, 0), 2), 1, round(E
  ) +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    axis.text.x = element_text(angle = 45, hjust = 1)
  ) +
```

```
ggtitle("Arcsine Transformation Comparison")
```

Effect size plot

R Code Start Over Run Code

```
effect_size_plot
```


Arcsine Transformation Comparison
Transformation
3
2.69
2.5
2
1.67
1.57
1
0.64
0.45
0
0
0.05
0.1
0.25
0.5
0.55
0.75
0.9
0.95
1
CTR

Some notes on this transformation

In the previous plot, note that the x-axis shows three equally-spaced δ_{CTR} segments (denoted by vertical red dashed lines).
The arcsine transformation will allocate:

- a smaller transformed effect size on the y-axis between two CTRs (denoted by horizontal purple dashed lines) as we get close to the middle end on the x-axis (i.e., 0.5),
- and larger transformed effect sizes on the y-axis as we get closer to the x-axis extremes (either 0 or 1).

Moreover

Note that the transformed effect size on the y-axis between
$\delta_{\text{CTR}} = \text{CTR}_B - \text{CTR}_A = 0.1 - 0.05 = 0.05$
is equal to the one from
$\delta_{\text{CTR}} = \text{CTR}_B - \text{CTR}_A = 0.95 - 0.9 = 0.05.$

Why is this effect behaviour happening in the above plot?

This is heavily related to the statistic's standard error of the corresponding two-proportion hypothesis Z-test.

Review

For a binomial proportion p, the variance is $\frac{p(1-p)}{n}$. This means:

- Proportions near 0.5 have the highest variance
- Proportions near 0 or 1 have low variance

> So, the distribution is heteroscedastic (variance not constant)

Code in R!

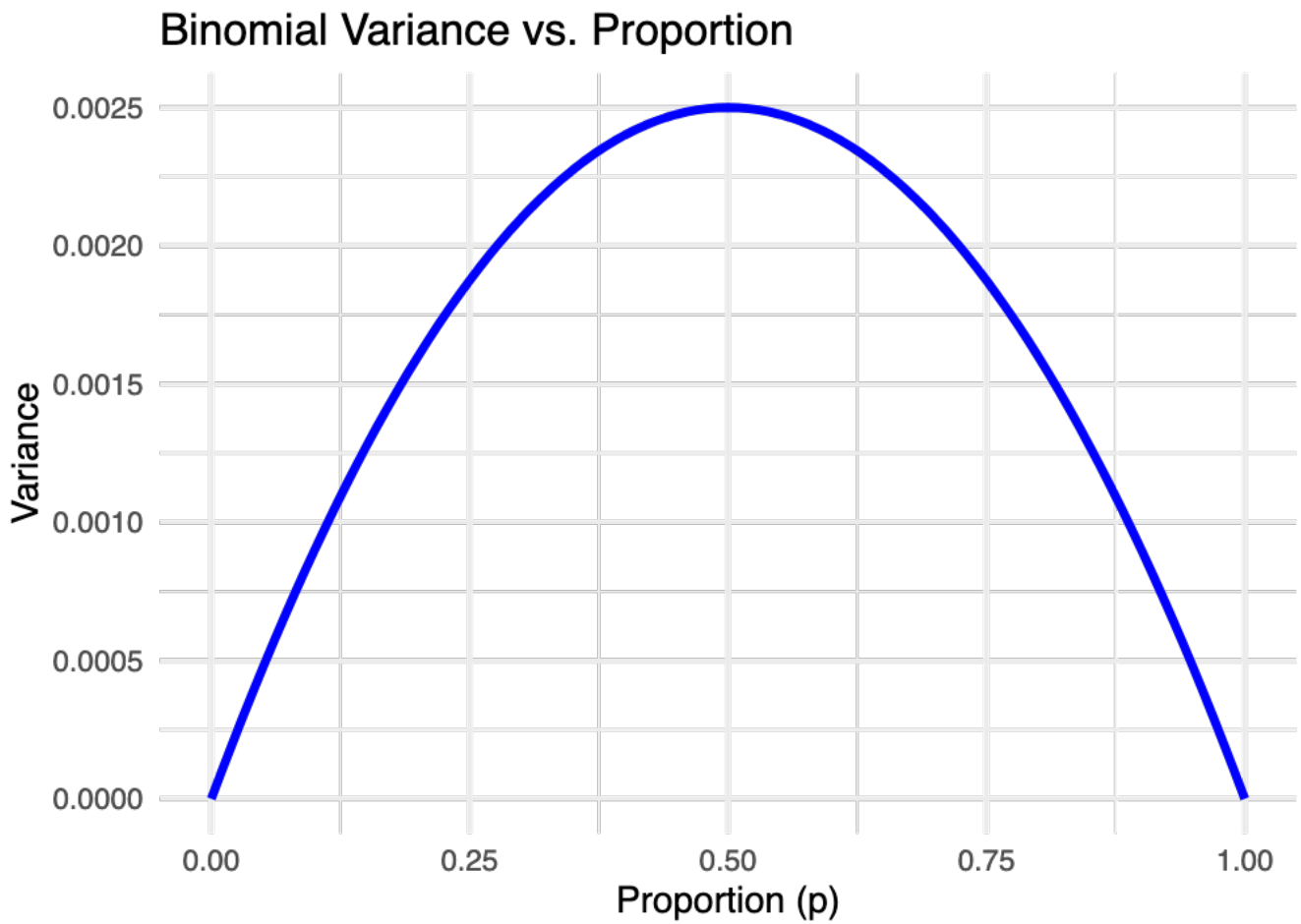
R Code Start Over Run Code

```r

# Set value of n and create a sequence of proportions between 0 and 1
n <- 100
p <- seq(0, 1, by = 0.01)

# Compute variance
variance <- p * (1 - p) / n

# Create a data frame
df <- data.frame(p = p, variance = variance)

prop_vs_variance <- ggplot(df, aes(x = p, y = variance)) +
  geom_line(color = "blue", linewidth = 1.2) +
  labs(
    title = "Binomial Variance vs. Proportion",
    x = "Proportion (p)",
    y = "Variance"
  ) +
  theme_minimal(base_size = 14)
```

Proportion vs Variance

R Code Start Over Run Code

```
prop_vs_variance
```


Binomial Variance vs. Proportion
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
Variance
0.00
0.25
0.50
0.75
1.00
Proportion (p)

For our previous practical example

We want to assess whether an experimental tweak in the website would increase from $\mathrm{CTR}_A = 0.005$ to $\mathrm{CTR}_B = 0.007$.
This means:
$\delta_{\mathrm{CTR}} = \mathrm{CTR}_B - \mathrm{CTR}_A = 0.002$;
with the following hypotheses
$H_0 : \delta_{\mathrm{CTR}} < 0.002$
$H_a : \delta_{\mathrm{CTR}} \geq 0.002$.

Two-proportion Z-test

Note that this two-proportion Z-test has the following test statistic:

$$Z = \frac{\hat{\delta}_{\text{CTR}} - 0.002}{\sqrt{\frac{\hat{\text{CTR}}_A(1-\hat{\text{CTR}}_A)}{n/2} + \frac{\hat{\text{CTR}}_B(1-\hat{\text{CTR}}_B)}{n/2}}} = \frac{\hat{\delta}_{\text{CTR}} - 0.002}{\text{SE}\left(\hat{\delta}_{\text{CTR}}\right)}.$$

The behaviour of SE

Now this standard error $\mathrm{SE}\left(\hat{\delta}_{\mathrm{CTR}}\right)$ will behave differently depending on where we locate our baseline CTR_A on the $[0, 1]$ range, even with the same δ_{CTR}!

Different baseline values

Let us plot the behaviour of $\mathrm{SE}\left(\hat{\delta}_{\mathrm{CTR}}\right)$ under different scenarios with a baseline of
$\mathrm{CTR}_A = 0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95$
An overall sample size of $n = 50, 100, 200, 500, 1000$.

> Heads-up: In each scenario, we consider a $\delta_{\mathrm{CTR}} = \mathrm{CTR}_B - \mathrm{CTR}_A = 0.05$.

Code in R!

R Code Start Over Run Code

```
CTR_A <- c(0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95)
overall_n <- c(50, 100, 200, 500, 1000)
SE_grid <- expand.grid(CTR_A, overall_n)
colnames(SE_grid) <- c("CTR_A", "overall_n")
SE_grid <- SE_grid %>%
  mutate(CTR_B = CTR_A + 0.05, SE = sqrt(((CTR_A * (1 - CTR_A)) / (overall_n /
```

Standard error vs CTR_A

R Code Start Over Run Code

```r
head(SE_grid, n=10)
```

```
   CTR_A overall_n CTR_B         SE
1   0.05        50   0.1 0.07416198
2   0.15        50   0.2 0.10723805
3   0.25        50   0.3 0.12609520
4   0.35        50   0.4 0.13674794
5   0.45        50   0.5 0.14106736
6   0.55        50   0.6 0.13964240
7   0.65        50   0.7 0.13228757
8   0.75        50   0.8 0.11789826
9   0.85        50   0.9 0.09327379
10  0.95        50   1.0 0.04358899
```

Code in R!

Let us check how this standard error graphically behaves across different baseline CTR_A by overall sample size n:

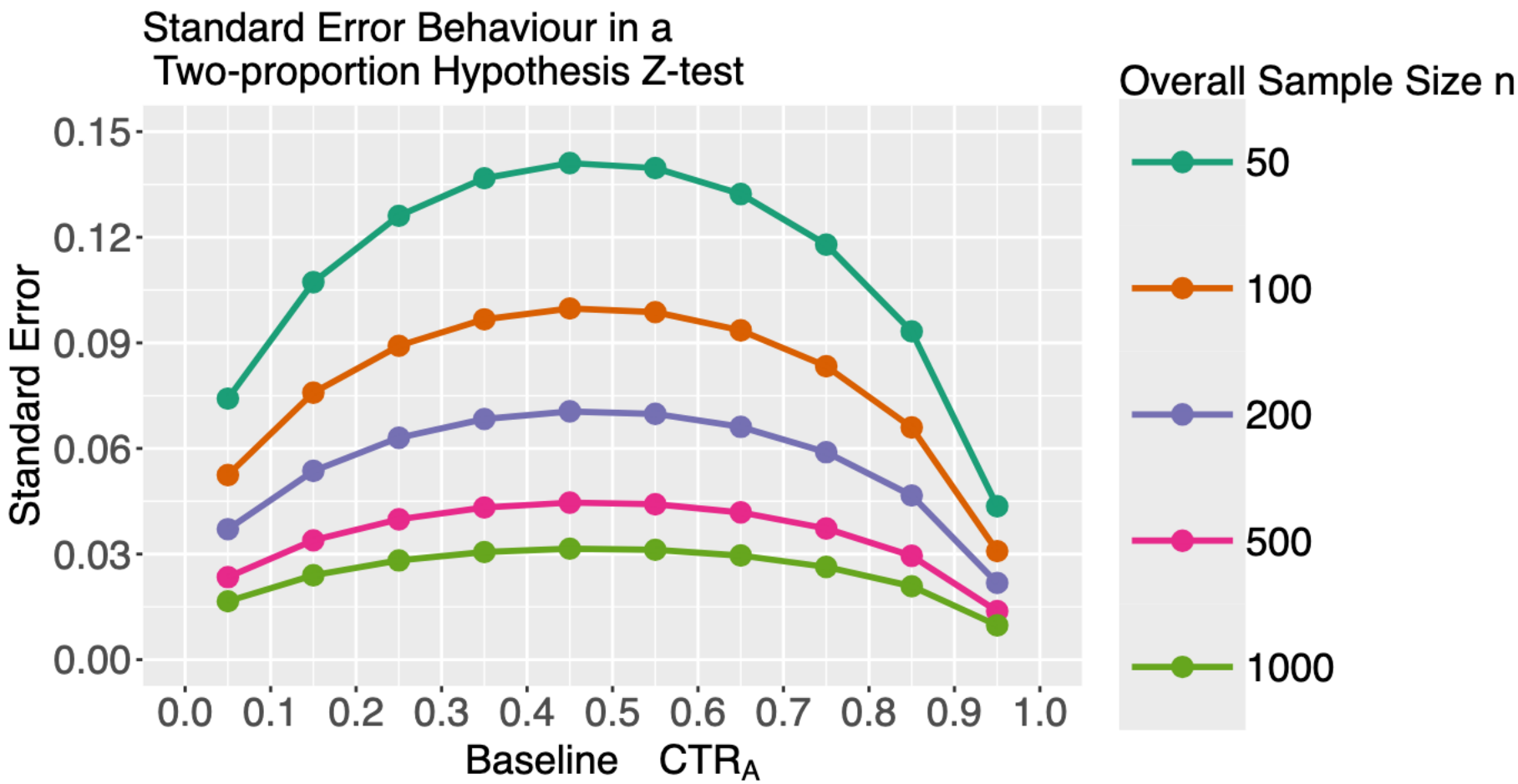
R Code Start Over Run Code

```r
SE_plot <- ggplot(SE_grid, aes(x = CTR_A, y = SE, colour = as.factor(overall_
  geom_point(size = 4) +
  geom_line(linewidth = 1) +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    strip.text = element_text(size = 18),
    legend.title = element_text(size = 18, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 18, margin = margin(r = 1, unit = "cm")
    legend.key.size = unit(2, "cm")
  ) +
  labs(x = expression("Baseline"~CTR[A]), y = "Standard Error") +
  labs(colour = "Overall Sample Size n") +
  ggtitle("Standard Error Behaviour in a \n Two-proportion Hypothesis Z-test"
  scale_x_continuous(
    limits = c(0, 1), breaks = seq(0, 1, 0.1),
  ) +
  scale_y_continuous(
    limits = c(0, 0.15), breaks = seq(0, 0.15, 0.03)
  ) +
  scale_color_brewer(palette = "Dark2")
```

Plot SE vs CTR_A

R Code Start Over Run Code

```
SE_plot
```


Standard Error Behaviour in a
Two-proportion Hypothesis Z-test
Standard Error
0.15
0.12
0.09
0.06
0.03
0.00
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
Baseline CTR$_A$
Overall Sample Size n
50
100
200
500
1000

Main takeaway

What is the main takeaway from this standard error plot?

We can see that the standard error of the test statistic approaches a maximum point around the middle-end of the range $[0, 1]$ on the x-axis.

Furthermore, if we increase the overall sample size n, we can bend the standard error curve.

Conclusion

Having said all this, we can state the following:

- A smaller transformed effect size in CTR Power Analysis would be associated with larger standard errors and CTRs jointly around 0.5.
- A larger transformed effect size in CTR Power Analysis would be associated with smaller standard errors and CTRs jointly closer to the extremes 0 or 1.

2.2. Using Function `pwr.2p.test()`

As in any sample size computation, we need to set up the significance level α and power $1 - \beta$:

R Code Start Over Run Code

```
alpha <- 0.05
pow <- 0.8
```

Arguments in `pwr.2p.test`!

Function `pwr.2p.test()` is part of the library `pwr`. It needs the following arguments:

- `h`: Desired effect size using the arcsine transformation `CTR_effect_size`.
- `sig.level`: Chosen α.
- `power`: Chosen power $1 - \beta$.
- `alternative`: Our hypothesis test is one-sided, thus `greater`.

The function will return the required sample size by treatment (i.e., the required number of impressions by treatment).

How to use `pwr.2p.test` function in R!

R Code Start Over Run Code

```r
CTR_test <- pwr.2p.test(
  h = CTR_effect_size,
  sig.level = alpha,
  power = pow,
  alternative = "greater"
)
CTR_test
```

```
     Difference of proportion power calculation for binomial distribution (arcsine transformation) 

              h = 0.02598837
              n = 18307.96
      sig.level = 0.05
          power = 0.8
    alternative = greater

NOTE: same sample sizes
```

Sample Size Analysis

Once we have checked how to use `pwr.2p.test()`, suppose your boss requires a sample size computation analysis with eight different control CTRs for treatment A:

R Code Start Over Run Code

```
n_lev <- 8
CTR_control_A <- (1:n_lev) / 200
CTR_control_A
```

```
[1] 0.005 0.010 0.015 0.020 0.025 0.030 0.035 0.040
```

Moreover

They also want to evaluate the impact of eight different CTR increases δ_{CTR} when using treatment B.

R Code Start Over Run Code

```
CTR_delta <- c((1:n_lev)/400)
CTR_delta
```

```
[1] 0.0025 0.0050 0.0075 0.0100 0.0125 0.0150 0.0175 0.0200
```

Therefore, for a single α and power $1 - \beta$, there will be 64 possible combinations of settings, as shown below in `CTR_experimental_grid`.

Code in R!

R Code Start Over Run Code

```r
CTR_experimental_grid <- expand.grid(CTR_control_A, CTR_delta)
colnames(CTR_experimental_grid) <- c("CTR_control_A", "CTR_delta")
CTR_experimental_grid$sample_size <- NA

for (i in 1:nrow(CTR_experimental_grid)) {
  CTR_experimental_grid[i, "sample_size"] <- 2 * ceiling(pwr.2p.test(
    h = ES.h(
      CTR_experimental_grid[i, "CTR_control_A"] +
        CTR_experimental_grid[i, "CTR_delta"],
      CTR_experimental_grid[i, "CTR_control_A"]
    ),
    sig.level = alpha,
    power = pow,
    alternative = "greater"
  )$n)
}
```

Results for Sample Size Analysis

R Code Start Over Run Code

```r
head(CTR_experimental_grid, n=15)
```

```
   CTR_control_A CTR_delta sample_size
1          0.005    0.0025       24330
2          0.010    0.0025       43880
3          0.015    0.0025       63162
4          0.020    0.0025       82226
5          0.025    0.0025      101086
6          0.030    0.0025      119742
7          0.035    0.0025      138196
8          0.040    0.0025      156452
9          0.005    0.0050        7154
10         0.010    0.0050       12090
11         0.015    0.0050       16922
12         0.020    0.0050       21692
13         0.025    0.0050       26402
14         0.030    0.0050       31060
15         0.035    0.0050       35666
```

How do we communicate this whole analysis?

We have huge sample sizes above $100,000$ and others below $5,000$.
The answer will be in a clever plotting arrangement!
The following plot shows the eight values of:
$\delta_{\text{CTR}} = \text{CTR}_B - \text{CTR}_A$
on the x-axis.

Moreover

The y-axis for sample size n has a primary logarithmic scale on the base 10 on the left-hand side, and the original scale for n on the right-hand side.
Finally, the legend on a colour scale incorporates the different control CTRs for treatment A by δ_{CTR} value.

> Heads-up: This plotting arrangement on the y-axis allows us to compare sample sizes with DIFFERENT ORDERS OF MAGNITUDE!

Code in R!

R Code Start Over Run Code

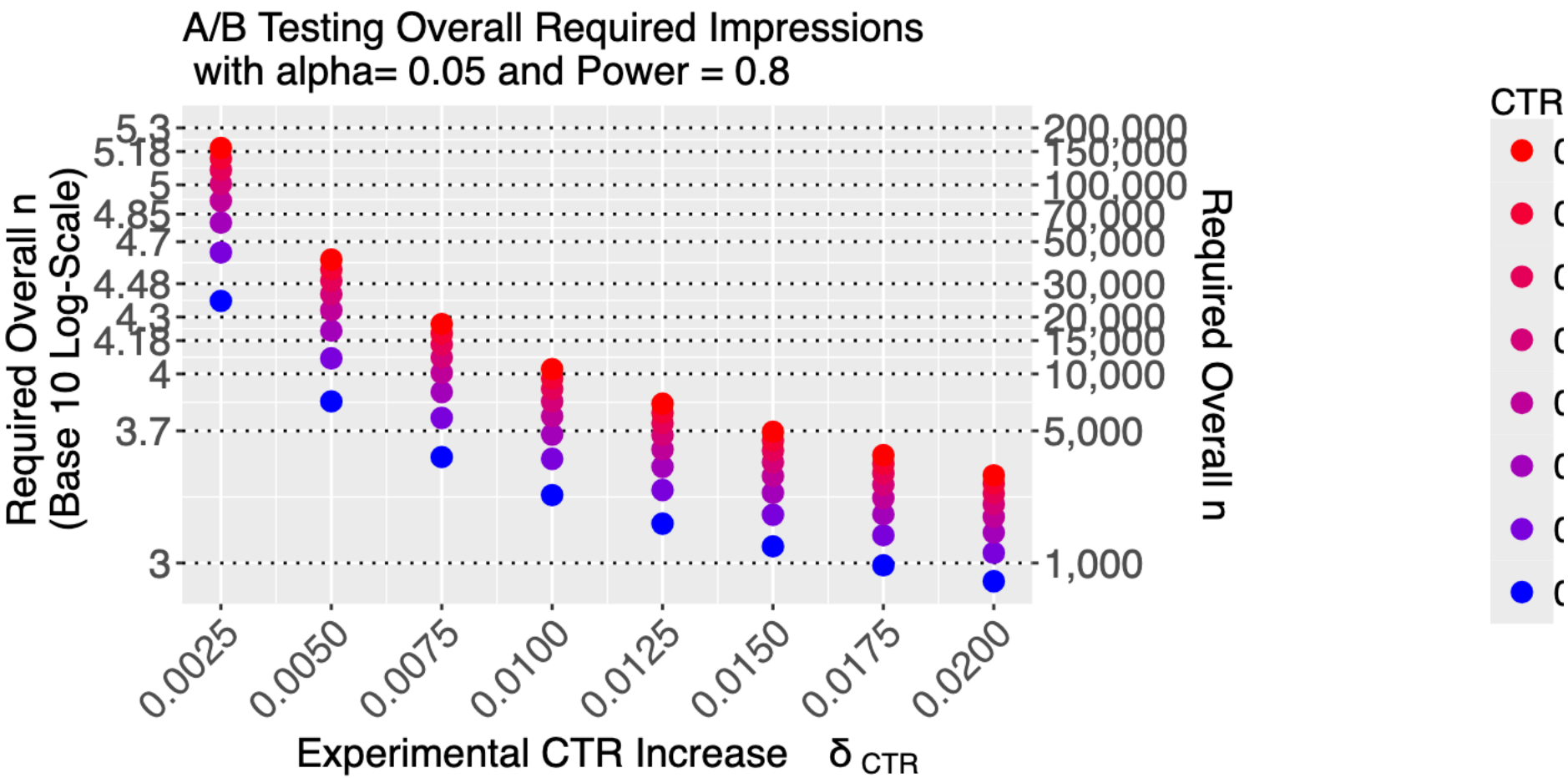
```r
colour_scale <- scales::seq_gradient_pal("blue", "red", "Lab")(seq(0, 1, leng
breaks_y_axis <- c(
  1000, 5000, 10000, 15000, 20000,
  30000, 50000, 70000, 100000, 150000, 200000
)

CTR_sample_size_plot <- CTR_experimental_grid %>%
  ggplot(aes(x = CTR_delta, y = sample_size, colour = as.factor(CTR_control_A
  geom_hline(yintercept = breaks_y_axis, linetype = "dotted") +
  geom_point(size = 4) +
  scale_y_continuous(
    trans = "log10", breaks = breaks_y_axis, labels = round(log10(breaks_y_ax
    sec.axis = sec_axis(~.,
      name = "Required Overall n", breaks = breaks_y_axis,
      labels = scales::comma(breaks_y_axis)
    )
  ) +
  xlab(expression("Experimental CTR Increase"~delta["CTR"])) +
  ylab("Required Overall n \n (Base 10 Log-Scale)") +
  scale_colour_manual(values = colour_scale, guide = guide_legend(reverse = T
  labs(colour = "CTR Control A") +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "right",
```

```
    legend.title = element_text(size = 16, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 16, margin = margin(r = 3, unit = "cm")
    legend.key.size = unit(1, 'cm'),
    legend.box.margin = margin(100, 100, 100, 100)
  ) +
  ggtitle("A/B Testing Overall Required Impressions \n with alpha= 0.05 and P
  scale_x_continuous(breaks = CTR_delta)
```

CTR sample size plot

R Code Start Over Run Code

```
CTR_sample_size_plot
```


A/B Testing Overall Required Impressions
with alpha= 0.05 and Power = 0.8
Required Overall n
(Base 10 Log-Scale)
5.3
5.18
5
4.85
4.7
4.48
4.3
4.18
4
3.7
3
200,000
150,000
100,000
70,000
50,000
30,000
20,000
15,000
10,000
5,000
1,000
Required Overall n
0.0025
0.0050
0.0075
0.0100
0.0125
0.0150
0.0175
0.0200
Experimental CTR Increase δ CTR
CTR

What is the main takeaway from this sample size plot?

We are seeing the arcsine transformation in action here for the effect size via the baseline CTR_A. It is susceptible to this baseline CTR_A but also to small increases δ_{CTR}.
Where are we standing with these CTR results?
Note that we might be running the experiment for a long time. Therefore, can we stop the A/B testing earlier once we see a significant result?

> Let us explore early stopping.

3. Early Stopping in A/B Testing

We will begin with a sample size computation to elaborate on early stopping in A/B testing.
Suppose our current site (i.e., control treatment A) has a CTR_A of 3%.
Our team has developed a tweak website layout (i.e., experimental treatment B).

> We are very optimistic that the tweak to the site could get us to 4.5% CTR, making a solid business case.

Overall sample size

Before conducting the corresponding A/B testing, we will power it accordingly via `pwr.2p.test()`.

R Code Start Over Run Code

```
control_ctr <- 0.03
tweak_ctr <- 0.045
alpha <- 0.05
pow <- 0.8

n <-  pwr.2p.test( h = ES.h(tweak_ctr, control_ctr),
                   sig.level = alpha, power = pow, alternative = "greater")$n

n_AB <- 2 * ceiling(n)
n_AB
```

```
[1] 3930
```

3.1. Introduction to Peeking

We obtain an OVERALL sample size of $n = 3930$. That is a lot of visitors!
Now, if we are running this experiment online while randomizing the experimental units in real-time, we presumably have access to the data as it is being collected. So we can check the state of our data collection as much as we want throughout the whole A/B testing.

> This is called PEEKING!

Moreover

If you can peek at the data as much as you want in real-time, you might be tempted to EARLY STOP the A/B testing.
Hence, if we keep performing the corresponding hypothesis testing every time we peek at the process, can we stop the experiment earlier before collecting the total sample size $n = 3930$?

Heads-up

Peeking in A/B testing still involves computing your overall sample size, call it $n_{\max}$, before starting the experiment.

If at some peek your UPDATED test statistic hits significance before we acquire all $n_{\max}$ experimental units, then A/B testing randomization and data collection will stop.

Early stop

By appealing to the previous logic, as soon as we are convinced that the experimental version of the site is more profitable than the control, we early stop the A/B testing and serve the experimental version to all visitors.
Again, simulation will be our friend to illustrate and practice the concept of peeking.
Function `sim_monitor_track()` simulates an A/B testing for comparing CTRs with two website versions.

Steps in `sim_monitor_track` function!

- The function sequentially generates independent visitors/impressions (i.e., experimental units), one by one, and randomly allocates them to either treatment A (control) or B (experimental tweak).
- Moreover, it generates the experimental unit's response (click or non-click) based on the population CTRs: CTR_A and CTR_B.
- Once each experimental unit has been recruited and their data has been collected, we compute/update the corresponding z-statistic and evaluate whether we reach significance or not (i.e., we conclude that $\mathrm{CTR}_B > \mathrm{CTR}_A$ at a significance level α).
- If we reach significance, the A/B testing will stop before reaching the projected overall sample size $n_{\max}$.

Arguments in `sim_monitor_track` function!

This function needs three arguments:

- `n_max` (projected maximum overall sample size $n_{\max}$),
- `CTR_A` (population CTR_A), and
- `CTR_B` (population CTR_B).

`sim_monitor_track` function in R!

R Code Start Over Run Code

```r
sim_monitor_track <- function(n_max, CTR_A, CTR_B) {
  # Significance level in A/B testing
  alpha <- 0.05

  # Data to record in each iteration to compute the z-statistic
  data <- matrix(0, 2, 2)
  z <- 0 # No test statistic at the beginning of the A/B testing
  n <- 0 # No visitors at the beginning of the A/B testing

  # Recordings of z statistics throughout the A/B testing
  z_opt <- rep(NA, n_max)

  # We will recruit A/B testing visitors within the loop
  # The experiment will last while we do not have a significant result
  # AND we do not reach the maximum number of experimental units (n_max)
  while ((z <= qnorm(1 - alpha)) && (n < n_max)) {
    n <- n + 1 # Increasing the number of recruited visitors

    # Randomize the visitor to treatment A (0) or B (1)
    x <- sample(0:1, size = 1)
    # See if they click on the link (1) or not (0)
    y <- rbinom(1, size = 1, prob = (1 - x) * CTR_A + x * CTR_B)

    # Update data table and recompute z-statistic
    data[x + 1, y + 1] <- data[x + 1, y + 1] + 1

    # Computing z-statistic
```

```
    if (min(data) > 0) {
      z <- (log(data[1, 1]) + log(data[2, 2]) - log(data[1, 2]) - log(data[2,
        sqrt(sum(1 / data))
    }
    z_opt[n] <- z # Recording new z-statistic
  }
  return(list(z = z_opt, n = n))
}
```

Run the function 4 times

We will execute this function 4 times (i.e., we will simulate four A/B testings). Their outputs will be the computed z-statistic sequences throughout each experiment.

We will use `n_max = 3930`, `CTR_A = 0.03`, and `CTR_B = 0.045`.

Run the simulation!

R Code Start Over Run Code

```
set.seed(554)

n_sim <- 4

z_sim <- matrix(NA, n_AB, n_sim)

for(m in 1:n_sim){
  z_sim[, m] <- sim_monitor_track(n_max = 3930, CTR_A = 0.03, CTR_B = 0.045)$z
}
```

Then

We wrangle our simulation results in the data frame `tracking_results`.

R Code Start Over Run Code

```
tracking_results <- data.frame(
  n = rep(1:n_AB, times = n_sim),
  sim = rep(1:n_sim,
    each = n_AB
  ),
  z = c(z_sim)
)
```

z-statistics

The next plot illustrates the sequence of z-statistics we computed every time we peek at the A/B testing once we got a new visitor, randomized them, and recorded their response.

The simulation sequences are coloured by each of the four replicates (a replicate is equivalent to one execution of `sim_monitor_track()`).

Note there is a horizontal dashed line indicating the cutoff $z_{1-\alpha} = 1.64$ with $\alpha = 0.05$. Recall that we are continuously performing one-sided tests.

Hence, when the computed test statistic $z_{\text{test}} > z_{1-\alpha}$, we have SUPPOSEDLY collected ENOUGH evidence to infer that $\text{CTR}_B > \text{CTR}_A$ with a significance level α.

Code in R!

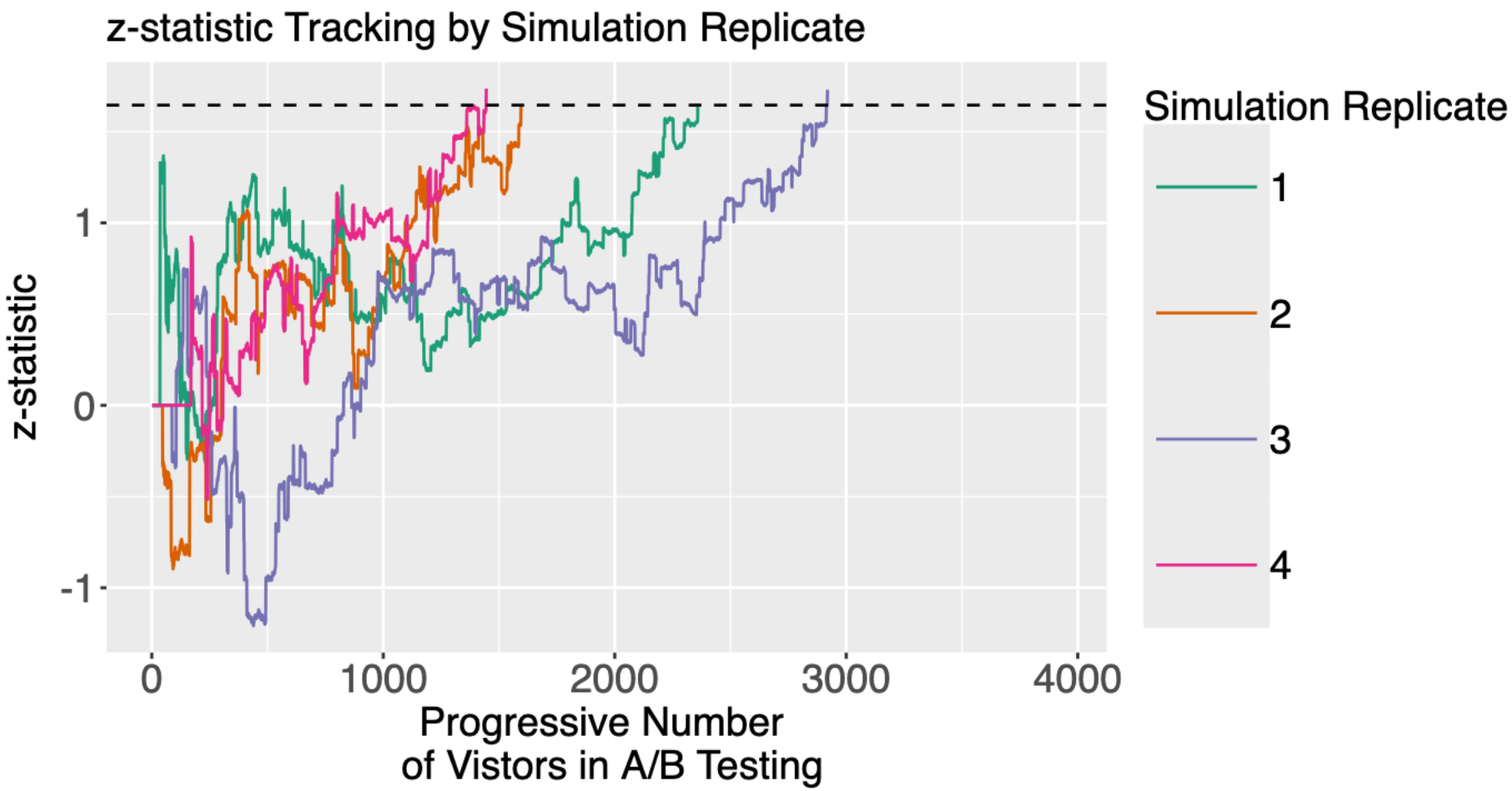
R Code Start Over Run Code

```
tracking_plot <- tracking_results %>% ggplot(aes(x = n, y = z, colour = as.fa
  geom_line() +
  geom_hline(yintercept = qnorm(1-alpha), linetype = "dashed") +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    legend.position = "right",
    legend.title = element_text(size = 18, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 18, margin = margin(r = 3, unit = "cm")
    legend.key.size = unit(2, 'cm'),
  ) +
  xlab("Progressive Number \n of Vistors in A/B Testing") +
  ylab("z-statistic") +
  labs(colour = "Simulation Replicate") +
  ggtitle("z-statistic Tracking by Simulation Replicate") +
  scale_color_brewer(palette = "Dark2")
```

z-statistic tracking plot

R Code Start Over Run Code

```r
options(warn = -1)
tracking_plot
```


z-statistic Tracking by Simulation Replicate
z-statistic
1
0
-1
0
1000
2000
3000
4000
Progressive Number
of Vistors in A/B Testing
Simulation Replicate
1
2
3
4

What is the main takeaway from this z-statistic tracking plot?

Under this population scenario, the four simulation replicates lead us to gather evidence to conclude that
$\mathrm{CTR}_B > \mathrm{CTR}_A$,
given that eventually our computed $z_{\text{test}} > z_{1-\alpha}$ (i.e., the z-statistic sequences cross the cutoff horizontal dashed line).

Note

It MIGHT be beneficial to end the A/B testing before reaching the maximum planned overall sample size n_{max} WHILE PEEKING.
However, let us dig more into this matter!

3.2. Three Simulation Studies with Aggressive Peeking

Before putting the previous appealing peeking idea into practice, we would need to run three simulation studies to check some key metrics.
These studies involve what we call aggresive peeking (i.e., when we check our data and test statistic every time we have a new experimental unit).

Thus, let us run three whole simulation studies. The function `sim_monitor()` does the same job as `sim_monitor_track()`, both require the same arguments.
However, `sim_monitor()` will only return the final z_{test} for a given A/B testing simulation replicate and the number of visitors at which we reach significance that will be less or equal than `n_max`.

Code in R!

R Code Start Over Run Code

```r
sim_monitor <- function(n_max, CTR_A, CTR_B) {
  # Significance level in A/B testing
  alpha <- 0.05

  # Data to record in each iteration to compute the z-statistic
  data <- matrix(0, 2, 2)
  z <- 0 # No test statistic at the beginning of the A/B testing
  n <- 0 # No visitors at the beginning of the A/B testing

  # We will recruit A/B testing visitors within the loop
  # The experiment will last while we do not have a significant result
  # AND we do not reach the maximum number of experimental units (n_max)
  while ((z <= qnorm(1 - alpha)) && (n < n_max)) {
    n <- n + 1 # Increasing the number of recruited visitors

    # Randomize the visitor to treatment A (0) or B (1)
    x <- sample(0:1, size = 1)
    # See if they click on the link (1) or not (0)
    y <- rbinom(1, size = 1, prob = (1 - x) * CTR_A + x * CTR_B)

    # Update data table and recompute z-statistic
    data[x + 1, y + 1] <- data[x + 1, y + 1] + 1

    # Recomputing z-statistic
    if (min(data) > 0) {
      z <- (log(data[1, 1]) + log(data[2, 2]) - log(data[1, 2]) - log(dat
        sqrt(sum(1 / data))
```

```
    }
  }
  return(list(z = z, n = n))
}
```

3.2.1. Case when $\mathrm{CTR}_B > \mathrm{CTR}_A$

What happens when $\mathrm{CTR}_B > \mathrm{CTR}_A$ in a simulation study if we apply aggressive peeking everytime there is a new visitor?
We will use the arguments `n_max = 3930`, `CTR_A = 0.03`, and `CTR_B = 0.045` in `sim_monitor()` with `1000` replicates for our simulation study.

Run the simulation!

R Code Start Over Run Code

```r
set.seed(554)

m_rep <- 1000 # Number of replicates

results_1 <- list(n = rep(NA, m_rep), z = rep(NA, m_rep)) # List of results

for (i in 1:m_rep) {
  sim <- sim_monitor(n_max = 3930, CTR_A = 0.03, CTR_B = 0.045)
  results_1$z[i] <- sim$z
  results_1$n[i] <- sim$n
}
```

Proportions of z-Statistics by Sample Size

Under this scenario, we sort the `1000` values corresponding to those effective sample sizes at which the A/B testing was stopped from the smallest to the largest. Then, we make seven bins. In each bin, we obtain the proportion of z-statistics that were less or equal than the cutoff $z_{1-\alpha} = 1.64$ with $\alpha = 0.05$ (column `(-Inf,1.64]`) and that proportion larger than $z_{1-\alpha}$ (column `(1.64, Inf]`).

Code in R!

R Code Start Over Run Code

```
sim_study_results_1 <- table(
  cut(results_1$n, breaks = c(0, 500, 1000, 1500, 2000, 2500, 3000, 3930), dig
  cut(results_1$z, breaks = c(-Inf, qnorm(1 - alpha), Inf))
) / m_rep

sim_study_results_1 <- format(sim_study_results_1)
```

Simulation Study Results

R Code Start Over Run Code

```r
sim_study_results_1
```

```
            (-Inf,1.64] (1.64, Inf]
(0,500]     "0.000"     "0.338"
(500,1000]  "0.000"     "0.207"
(1000,1500] "0.000"     "0.119"
(1500,2000] "0.000"     "0.091"
(2000,2500] "0.000"     "0.067"
(2500,3000] "0.000"     "0.038"
(3000,3930] "0.087"     "0.053"
```

What is the main takeaway from this simulation study in the context of this population?

The expression $\mathrm{CTR}_B > \mathrm{CTR}_A$ corresponds to our alternative hypothesis H_a. We know this hypothesis is in fact true since the population $\mathrm{CTR}_A = 0.03$ and $\mathrm{CTR}_B = 0.045$.

When $z_{\text{test}} \leq z_{1-\alpha}$, we fail reject the null hypothesis H_0 which is committing type II error!

Conclusion

This table is telling us that the observed power in this simulation study is $1 - \beta = 1 - 0.087 = 0.913$!

It seems that an aggresive peeking improves the power of the test when, in fact, $\mathrm{CTR}_B > \mathrm{CTR}_A$.

However, note that the increase from CTR_A to CTR_B is 50%.

What would happen if the percentage CTR increase is smaller?

3.2.2. Case when $\mathrm{CTR}_B > \mathrm{CTR}_A$

What happens when $\mathrm{CTR}_B > \mathrm{CTR}_A$ under a smaller percentage increase in a simulation study if we apply aggresive peeking every time there is a new visitor? We will use the arguments `n_max = 3930`, `CTR_A = 0.03`, and `CTR_B = 0.036` in `sim_monitor()` with `1000` replicates for our simulation study. This is a CTR percentage increase of 20%.

Run the simulation!

R Code Start Over Run Code

```r
set.seed(554)

m_rep <- 1000 # Number of replicates

results_2 <- list(n = rep(NA, m_rep), z = rep(NA, m_rep)) # List of results

for (i in 1:m_rep) {
  sim <- sim_monitor(n_max = 3930, CTR_A = 0.03, CTR_B = 0.036)
  results_2$z[i] <- sim$z
  results_2$n[i] <- sim$n
}
```

R Error in `stopifnot(length(n) == 1L)`

```
Error: reached elapsed time limit
```

Then we obtain the seven-bin table as in the previous simulation study.

R Code Start Over Run Code

```
sim_study_results_2 <- table(
  cut(results_2$n, breaks = c(0, 500, 1000, 1500, 2000, 2500, 3000, 3930), dig
  cut(results_2$z, breaks = c(-Inf, qnorm(1 - alpha), Inf))
) / m_rep

sim_study_results_2 <- format(sim_study_results_2)
```

Simulation Study Results

R Code Start Over Run Code

```r
sim_study_results_2
```

```
            (-Inf,1.64] (1.64, Inf]
(0,500]     "0.000"     "0.124"
(500,1000]  "0.000"     "0.076"
(1000,1500] "0.000"     "0.052"
(1500,2000] "0.000"     "0.033"
(2000,2500] "0.000"     "0.027"
(2500,3000] "0.000"     "0.032"
(3000,3930] "0.311"     "0.042"
```

What is the main takeaway from this simulation study in the context of this population?

When there is an aggresive peeking in a population scenario under a small percentage CTR increase, then we see a concerning power decrease.
The power is $1 - \beta = 1 - 0.454 = 0.546$.

3.2.3. Case when $\mathrm{CTR}_B = \mathrm{CTR}_A$

What happens when $\mathrm{CTR}_B = \mathrm{CTR}_A$ in a simulation study if we apply aggressive peeking everytime there is a new visitor?
We will use the arguments `n_max = 3930`, `CTR_A = 0.03`, and `CTR_B = 0.03` in `sim_monitor()` with `1000` replicates for our simulation study.

R Code Start Over Run Code

```r
set.seed(554)

m_rep <- 1000 # Number of replicates

results_3 <- list(n = rep(NA, m_rep), z = rep(NA, m_rep)) # List of results

for (i in 1:m_rep) {
  sim <- sim_monitor(n_max = 3930, CTR_A = 0.03, CTR_B = 0.03)
  results_3$z[i] <- sim$z
  results_3$n[i] <- sim$n
}
```

R Error in `stopifnot(length(n) == 1L)`

```
Error: reached elapsed time limit
```

Then we obtain the seven-bin table as in the previous simulation study.

R Code Start Over Run Code

```
sim_study_results_3 <- table(
  cut(results_3$n, breaks = c(0, 500, 1000, 1500, 2000, 2500, 3000, 3930), dig
  cut(results_3$z, breaks = c(-Inf, qnorm(1 - alpha), Inf))
) / m_rep

sim_study_results_3 <- format(sim_study_results_3)
```

Simulation Study Results

R Code Start Over Run Code

```r
sim_study_results_3
```

```
            (-Inf,1.64] (1.64, Inf]
(0,500]     "0.000"     "0.064"
(500,1000]  "0.000"     "0.027"
(1000,1500] "0.000"     "0.018"
(1500,2000] "0.000"     "0.007"
(2000,2500] "0.000"     "0.007"
(2500,3000] "0.000"     "0.009"
(3000,3930] "0.398"     "0.010"
```

What is the main takeaway from this simulation study in the context of this population?

In this context, the proportion of replicates where $z_{\text{test}} > z_{1-\alpha}$ correspond to the type I error.
In this case, an aggresive peeking severely inflates the type I error $\alpha = 1 - 0.751 = 0.249$.

3.3. Final Simulation Studies with Principled Peeking

Recall that non-peeking and aggresive peeking are the following extremes in A/B testing:

- Accruing all $n_{\max}$ experimental units and not computing/looking at the test statistic until then (non-peeking).
- Real-time monitoring of the test statistic and stopping early (aggresive peeking).

Principled peeking

Both extremes might not be good ideas! Hence, is there something in between these two extremes?
A common A/B testing practice is principled peeking.
This practice involves defining a fixed number of peeks before starting the experiment. It has the following characteristics:

Steps for principled peeking

- Define an m number of principled peeks before starting the experiment.
- Principled peeks will begin at a certain collected number of recruited experimental units $n_{\min} > 0$.
- Principled peeks will be evenly spaced between experimental units $n_{\min}$ and $n_{\max}$.
- Use the Bonferroni correction to adjust α in our hypothesis testings such that α/m.

- If at a certain principled peek we reach significance before $n_{\max}$, then the A/B testing stops.
- If we never reach significance before $n_{\max}$, then the last principled peek will correspond to $n_{\max}$.

`sim_rej` function in R!

- Function `sim_rej()` executes a set of simulation CTR studies under a defined number of principled peeks (one study with `m_rep` replicates per principled peek).
- These peeks begin at the experimental unit `n_min`.
- The rest of the arguments are the same ones from our previous functions.

Code in R!

R Code Start Over Run Code

```r
sim_rej <- function(n_min, n_max, alpha = 0.05, CTR_A, CTR_B, n_peeks, m_rep)

  # Setting up overall study metrics by number of peek setting
  rej_mean <- rep(0, length(n_peeks))
  rej_sd <- rep(0, length(n_peeks))
  n_mean <- rep(0, length(n_peeks))
  n_sd <- rep(0, length(n_peeks))

  # Beginning studies
  # By number of principled peekings
  for (n in 1:length(n_peeks)) {

    z_cutoff <- qnorm(1 - alpha / n_peeks[n]) # Note the Bonferroni correctio

    # Visitor numbers at which peek happens
    n_inspect <- round(n_min + (n_max - n_min) * (1:n_peeks[n]) / n_peeks[n])

    # Individual simulation study metrics
    rej <- rep(0, m_rep)
    n_stop <- rep(0, m_rep)

    # Beginning individual simulation
    for (m in 1:m_rep) {

      # Test statistics to record
      zz <- rep(0, length(n_inspect))

```

```
    # Simulating the sequence of all visitors
    x <- sample(0:1, size = n_max, replace = TRUE)
    y <- rbinom(n_max, size = 1, prob = (1 - x) * CTR_A + x * CTR_B)

    # Executing planned hypothesis testings (by number of peeks)
    for (ni in 1:length(n_inspect)) {

      # Updating terms to compute test statistic
      np <- n_inspect[ni]
      x_new <- sum(x[1:np])
      x_old <- np - x_new
      y_1_new <- sum(x[1:np] * y[1:np])
      y_1_old <- sum((1 - x[1:np]) * y[1:np])

      p_star <- (y_1_new + y_1_old) / np
      z <- (y_1_new / x_new - y_1_old / x_old) / (sqrt(p_star * (1 - p_star
      if (is.nan(z)) {
        z <- 0
      }
      zz[ni] <- z
    }

    # Shall we stop the study at a given peek?
    rej[m] <- any(zz > z_cutoff) # +1 in metric if we reject the null hypot
    if (rej[m]) {
      n_stop[m] <- n_inspect[which(zz > z_cutoff)[1]]
    } else {
```

```
        n_stop[m] <- n_max
      }
    }

    # Computing overall study metrics by number of peeks settings
    rej_mean[n] <- mean(rej) # Proportion of times we rejected the null hypot
    rej_sd[n] <- sd(rej)
    n_mean[n] <- mean(n_stop) # Average number of visitors required for early
    n_sd[n] <- sd(n_stop)

  }
  return(list(rej_mean = rej_mean, rej_sd = rej_sd, n_mean = n_mean, n_sd = n
}
```

Setup the parameters

- We execute `sim_rej()` with the arguments below.
- Note we set up the following principled peeks: `1, 2, 5, 10, 15, 20, 25`.
- Our `n_min` parameter is `1000`.

Moreover

We run these studies under two realities:

- Reality 1: No change in CTRs between both treatments.
- Reality 2: Increase from $\mathrm{CTR}_A = 0.03$ to $\mathrm{CTR}_B = 0.045$.

Run the simulation!

R Code Start Over Run Code

```r
set.seed(321)

# Running reality 1
reality_1 <- sim_rej(
  n_min = 1000, n_max = 3930, CTR_A = 0.03, CTR_B = 0.03,
  n_peeks = c(1, 2, 5, 10, 15, 20, 25), m_rep = 1000
)

# Collecting type I error for Reality 1
reality_1_type_1_error <- tibble(n_peeks = c(1, 2, 5, 10, 15, 20, 25), type_1

# Running reality 2
reality_2 <- sim_rej(
  n_min = 1000, n_max = 3930, CTR_A = 0.03, CTR_B = 0.045,
  n_peeks = c(1, 2, 5, 10, 15, 20, 25), m_rep = 1000
)

# Collecting power for Reality 2
reality_2_power <- tibble(n_peeks = c(1, 2, 5, 10, 15, 20, 25), power = reali

# Collecting average recruited visitors by reality and number of peeks
avg_recruited <- tibble(
  Reality = as.factor(c(rep("Reality 1", 7), rep("Reality 2", 7))),
  n_peeks = rep(c(1, 2, 5, 10, 15, 20, 25), 2),
  avg_number = c(reality_1$n_mean, reality_2$n_mean)
)
```

Code in R!

R Code Start Over Run Code

```
avg_vistors_reality <- ggplot(avg_recruited, aes(x = n_peeks, y = avg_number,
  geom_point(size = 4) +
  geom_line(size = 1) +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    legend.position = "right",
    legend.title = element_text(size = 16, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 16, margin = margin(r = 1, unit = "cm")
    legend.key.size = unit(2, 'cm')
  ) +
  labs(x = "Number of Peeks in A/B Testing", y = "Average Recruited \n Visito
  scale_x_continuous(breaks = c(1, 2, 5, 10, 15, 20, 25), labels = c(1, 2, 5,
  ggtitle("") +
  scale_color_manual(values=c("red", "blue")) +
  coord_cartesian(ylim = c(2900, 4000)) +
  scale_y_continuous(breaks = seq(3000, 4000, 200))

type_1_error_reality_1 <- ggplot(reality_1_type_1_error, aes(x = n_peeks, y =
  geom_point(size = 4, colour = "red") +
  geom_line(size = 1, colour = "red") +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    legend.position = "right",
```

```
    legend.title = element_text(size = 16, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 16, margin = margin(r = 1, unit = "cm")
    legend.key.size = unit(2, 'cm')
  ) +
  labs(x = "Number of Peeks in A/B Testing", y = "Type I Error Rate") +
  coord_cartesian(ylim = c(0, 0.06)) +
  scale_y_continuous(breaks = seq(0, 0.06, 0.01)) +
  scale_x_continuous(breaks = c(1, 2, 5, 10, 15, 20, 25), labels = c(1, 2, 5,
  ggtitle("Reality 1: Type I Error Rate by Number of Peeks")

power_reality_2 <- ggplot(reality_2_power, aes(x = n_peeks, y = power)) +
  geom_point(size = 4, colour = "blue") +
  geom_line(size = 1, colour = "blue") +
  theme(
    plot.title = element_text(size = 18),
    axis.text = element_text(size = 18),
    axis.title = element_text(size = 18),
    legend.position = "right",
    legend.title = element_text(size = 16, margin = margin(r = 1, unit = "cm"
    legend.text = element_text(size = 16, margin = margin(r = 1, unit = "cm")
    legend.key.size = unit(2, 'cm')
  ) +
  labs(x = "Number of Peeks in A/B Testing", y = "Power") +
  coord_cartesian(ylim = c(0.4, 0.9)) +
  scale_y_continuous(breaks = seq(0.4, 0.9, 0.05)) +
  scale_x_continuous(breaks = c(1, 2, 5, 10, 15, 20, 25), labels = c(1, 2
  ggtitle("Reality 2: Power by Number of Peeks")
```

```
bottom_row <- plot_grid(type_1_error_reality_1, power_reality_2, align = 'h')
```

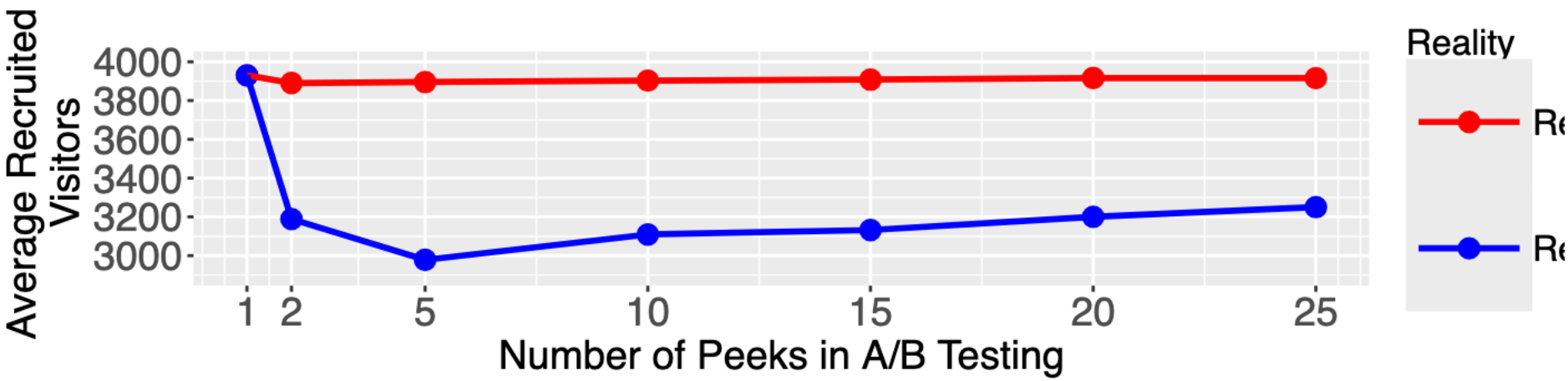
R Warning

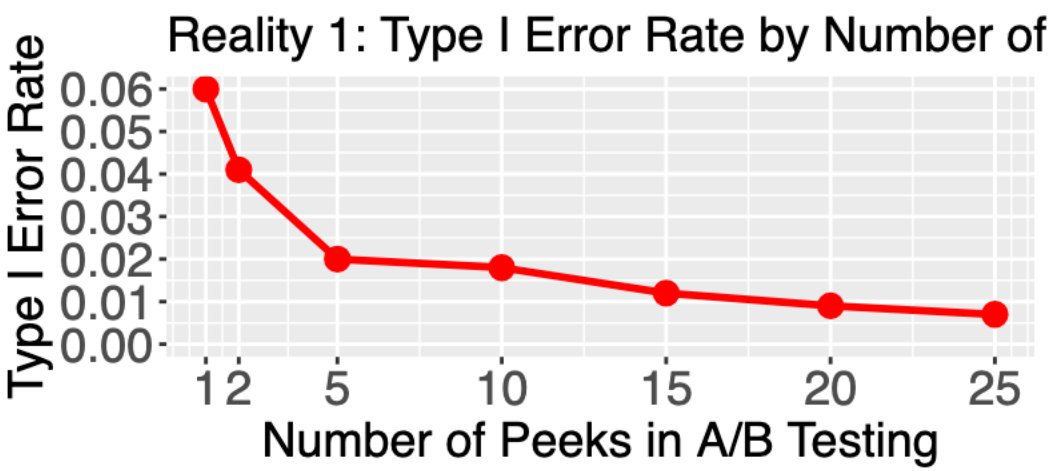
```
<warning/lifecycle_warning_deprecated>
Warning:
Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
ℹ Please use `linewidth` instead.
```

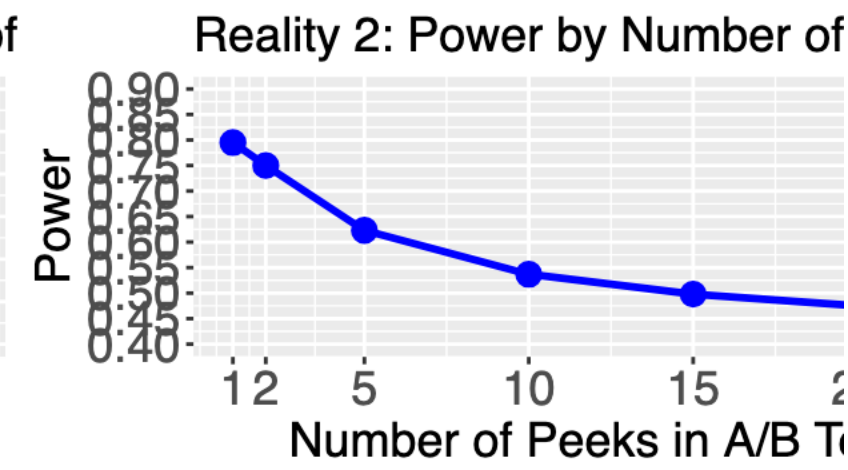
Plot in R!

R Code Start Over Run Code

```
plot_grid(avg_vistors_reality, bottom_row, nrow = 2)
```


Average Recruited Visitors
4000
3800
3600
3400
3200
3000
1
2
5
10
15
20
25
Number of Peeks in A/B Testing
Reality
Re
Re


Reality 1: Type I Error Rate by Number of
Type I Error Rate
0.06
0.05
0.04
0.03
0.02
0.01
0.00
1
2
5
10
15
20
25
Number of Peeks in A/B Testing


Reality 2: Power by Number of
Power
0.90
0.40
1
2
5
10
15
Number of Peeks in A/B T

What are the main takeaways from these plots?

- There is a decrease in recruited visitors when we have 5 principled peeks in Reality 2 ($\text{CTR}_B > \text{CTR}_A$). This does not happen in Reality 1 ($\text{CTR}_B = \text{CTR}_A$) at any number of principled peeks.
- The Bonferroni correction greatly prevents false positives as we increase the number of principled peeks in Reality 1.
- We need to be cautious when increasing the number of principled peeks in Reality 2 since this might impact the power. From these plots, 2 principled peeks are ideal!

4. Wrapping Up

- A clever plotting in Power Analysis is mandatory in A/B testing practice.
- Early stopping in A/B testing is an appealing idea, but we have to be statistically cautious.
- In aggressive peeking, we see “decent” power if indeed the experimental treatment B beats the control treatment A by a large amount. However, if both CTRs are close, then aggressive peeking plays against our power.
- In aggressive peeking, we will have type I error rate inflation if experimental treatment B is actually no better than control treatment A.
- Principled peeking involves planning our test checking and relies on the Bonferroni correction. It prevents type I error rate inflation if experimental treatment B is actually no better than control treatment A.

DSCI 554 - Experimentation and Causal Inference

Speaker notes